

Sistemas de recomendación para películas a partir de filtros colaborativos y basados en contenido

Juan Felipe Vargas Quintana

Sara Maria Cano Martínez

Minería de Datos para Grandes Volúmenes de Información

23 de junio de 2025

1. Introducción

Los sistemas de recomendación se han convertido en herramientas importantes en la industria. Esto se debe a la gran oportunidad que representan estos para las empresas, generando mayor valor, al brindarle al usuario lo que este desea o requiere. También, para los usuarios son de gran utilidad, ya que les permite invertir menos tiempo en búsquedas y obtener mejores resultados. El problema detrás de los sistemas de recomendación radica en la capacidad computacional requerida para la implementación de estos, desde el punto de vista de almacenamiento y la capacidad de cómputo que esto requiere. En este proyecto se desarrollará un sistema de recomendación de películas basado en filtrado colaborativo y filtrado basado en contenido, por medio del uso de arquitectura con AWS, teniendo un enfoque basado en la dimensionalidad de los datos.

2. Pregunta de Investigación y Objetivos

2.1. Pregunta de investigación

¿Cómo diseñar y evaluar un sistema de recomendación de películas que proporcione predicciones a partir de las calificaciones de los usuarios y el contenido de los items?

2.2. Objetivos

2.2.1. General

Desarrollar un sistema de recomendación de películas utilizando el conjunto de datos MovieLens.

2.2.2. Específicos

- Preparar el conjunto de datos MovieLens para su uso en modelos de recomendación.
- Implementar modelos de recomendación basados en filtrado colaborativo y filtrado basado en contenido.
- Obtener un RMSE menor a 0.9 para las calificaciones del modelo de filtrado colaborativo.
- Verificar la coherencia temática y de género en las recomendaciones del modelo de filtrado basado en contenido.

3. Estado del arte

En la literatura se encuentran numerosas implementaciones de sistemas de recomendación de películas ya que es un tema de gran interés para la industria del entretenimiento. Los autores en [1] se enfocan en un sistema recomendador de películas basado en el Filtrado Colaborativo (CF) utilizando el algoritmo Alternating Least Squares (ALS). Este algoritmo ALS es descrito como una de las técnicas más importantes para construir motores de recomendación de películas y un modelo de factorización matricial relacionado con CF, utilizado para predecir calificaciones de usuarios. También se menciona que ALS es un algoritmo paralelo simple diseñado para abordar problemas de escalabilidad con conjuntos de datos muy grandes y que fue implementado para el Netflix Prize. La metodología principal se centra en proponer un sistema recomendador de películas basado en ALS usando Apache Spark. La evaluación de este modelo se realiza utilizando métricas como el tiempo de ejecución, el error cuadrático medio (RMSE) de la predicción de calificaciones y el ranking. Los autores obtienen un RMSE óptimo de 0.9167 y un tiempo de ejecución de 1.07422 segundos.

Por otro lado, los autores en [6] desarrollan un sistema de recomendación de películas que se centra en el Filtrado Basado en Contenido (Content Based Filtering). El sistema procesa los datos de las películas utilizando la técnica Term-Frequency Inverse Document Frequency (TF-IDF) para la vectorización, creando un Modelo Espacio Vectorial (Vector Space Model). Para medir la similitud entre las películas, utilizan el método de Similitud del Coseno (Cosine Similarity). Además, proponen el uso de la Distancia de Levenshtein junto con la similitud del coseno como una medida de similitud híbrida para mejorar el rendimiento y la precisión. Los features extraídos para el contenido incluyen títulos, géneros, directores, actores y sinopsis. El sistema también incorpora el análisis de sentimientos de las reseñas de los usuarios, clasificándolas como positivas o negativas usando TF-IDF Vectorizer y un clasificador Multinomial Naive Bayes. La implementación utiliza Python con Flask como framework, HTML, CSS y JS para el frontend, obteniendo datos de la API de TMDb (*The Movie Database*) y mediante web scraping. La contribución principal que señalan los autores es la propuesta de un nuevo enfoque para mejorar el rendimiento del filtrado basado en contenido, particularmente para abordar problemas como el *cold*

start, mediante la mejora de la medida de similitud y la representación del contenido de la película. Afirman que el uso combinado de la Distancia de Levenshtein con Similitud de Coseno y TF-IDF en su sistema híbrido proporciona recomendaciones más precisas al usuario, mejorando la precisión en la representación de la película.

4. Metodología

4.1. Filtrado colaborativo (ALS)

El filtrado colaborativo es un método usado para realizar sistemas de recomendación a partir de las calificaciones que los usuarios le hacen a los ítems, en este caso películas. Basado en [8], se tiene la matriz de usuarios-ítems o de utilidad $R \in \mathbb{R}^{n \times m}$, donde n y m corresponden al número de usuarios e ítems, respectivamente. Las entradas de esta matriz corresponden a las calificaciones de los usuarios hacia los ítems. La idea detrás de este método es aproximar la matriz R a partir de dos matrices $U \in \mathbb{R}^{n \times k}$ y $M \in \mathbb{R}^{k \times m}$, donde $R \approx U \times M$. El número k , en la dimensión de las matrices, se conoce como número de factores latentes. Con los factores latentes se busca representar los usuarios y las películas por medio de un vector de características de dimensión k . Es decir, se hace una asignación de coordenadas en un espacio de menor dimensión. Aunque se tiene una correspondencia entre los vectores y los usuarios o ítems, el significado de las entradas en estos no tiene una interpretación cualitativa.

La otra parte del algoritmo consiste en estimar las matrices que aproximan a la de utilidad. Esto se logra a partir del método de mínimos cuadrados alternados. La idea es realizar un proceso de optimización alternante, donde se fija el valor de una matriz optimizando la otra y viceversa. La optimización se realiza respecto a una función objetivo y se detiene ante un criterio de parada.

4.2. Filtrado basado en contenido

El filtrado basado en contenido se basa en la idea de recomendar ítems a partir de la similaridad que existe entre ellos. Por medio de la similaridad entre los ítems se pueden plantear múltiples sistemas de recomendación. Para la implementación de este método se debe tener una forma de representar los ítems y una medida de similaridad a partir de esta representación. Uno de los métodos usados para este objetivo es el uso de la métrica TF-IDF. Esta métrica permite convertir texto en vectores. Teniendo esto en cuenta, su utilidad radica en convertir las etiquetas de las películas en vectores. Esto se logra poniendo pesos a las palabras de acuerdo con la unicidad de estas, capturando la relevancia entre el resto de palabras del documento [8]. Lo que se busca es ponderar la aparición de la palabra en el documento, pero a su vez con la frecuencia de esta en el resto de documentos. Así, el peso será proporcional a la frecuencia de la palabra en el documento e inversamente proporcional a la frecuencia en todos los documentos. Con esto, se puede hallar la similaridad entre películas a partir de una medida de similitud, como la de coseno. Así, valores cercanos a 1 implican mayor similaridad y hacia 0 poca similaridad.

4.3. Metodología de entrenamiento y evaluación

Los modelos se entrenan y validan sobre la partición de los datos procesados dividida en un **77 %** para el conjunto de entrenamiento y un **23 %** para el conjunto de prueba. Esta proporción logra un compromiso adecuado entre disponer de suficientes observaciones para ajustar los parámetros ($\approx 24,6$ M de ratings) y un subconjunto independiente lo bastante amplio ($\approx 7,4$ M) para medir la generalización.

El modelo de filtrado colaborativo se construyó con *ALS* en PySpark 3.5, empleando los hiperparámetros resumidos en la Tabla 1. Para el parámetro *rank* se evaluaron tres valores (10, 20 y 30) y cada variante se entrenó en un clúster AmazonEMR configurado con los pasos del tutorial disponible en GitHub sobre sistemas de recomendación de anime [7]. Esta infraestructura garantiza la reproducibilidad de los experimentos y permite escalar horizontalmente si aumenta el volumen de datos o se requieren nuevas pruebas. El recomendador basado en contenido reproduce la metodología de [6]: se genera un vector *TF-IDF* a partir del título y los géneros de cada película y la similitud entre pares se calcula mediante el coseno de sus vectores normalizados.

Cuadro 1: Parámetros utilizados en el modelo *ALS*.

Parámetro	Valor	Descripción
rank	{10, 20, 30}	Número de factores latentes explorados
maxIter	20	Iteraciones máximas del algoritmo
regParam	0.1	Coefficiente de regularización L2
nonnegative	True	Obliga que U y M a ser ≥ 0
coldStartStrategy	"drop"	Descarta predicciones NaN en usuarios/ítems no vistos

Para *ALS* la métrica de evaluación es el error cuadrático medio (*RMSE*), definido como

$$RMSE = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} (r_{ui} - \hat{r}_{ui})^2},$$

donde $\mathcal{T}$ es el conjunto usuario-película del *test set*. El *RMSE* se adopta porque penaliza con mayor severidad los errores grandes, es la métrica de uso predominante en este tipo de problemas y permite comparación directa con trabajos previos.

En cambio, el recomendador basado en contenido no dispone de una verdad numérica que marque la “similitud correcta” entre películas; la relevancia es subjetiva y dependiente del contexto. Métricas automáticas como *precision* o *recall* requerirían conjuntos de relevancia creados manualmente para decenas de miles de títulos, algo inviable. Por ello su evaluación se realiza mediante un análisis cualitativo: se escoge una película de referencia (p. ej. *Toy Story* 1995) y se comprueba si las sugerencias incluyen secuelas, producciones del mismo estudio, títulos de géneros coincidentes y una diversidad razonable de estilos. Esta inspección humana, aunque no genera un número único comparable, es la práctica aceptada cuando el modelo se apoya exclusivamente en metadatos textuales.

5. Análisis de los datos

El conjunto de datos MovieLens (ml-32m), obtenido de [5], contiene información sobre calificaciones y etiquetas de películas proporcionadas por usuarios en la plataforma MovieLens. El archivo `ratings.csv` contiene las calificaciones de los usuarios a las películas, con los siguientes campos: `userId`, `movieId`, `rating` y `timestamp`. Esta última columna no se tendrá en cuenta para los modelos. En la tabla 2 se observan los primeros 5 registros de esta base de datos.

Cuadro 2: Ejemplo de las primeras filas de `ratings.csv`.

<code>userId</code>	<code>movieId</code>	<code>rating</code>	<code>timestamp</code>
1	17	4.0	944249077
1	25	1.0	944250228
1	29	2.0	943230976
1	30	5.0	944249077
1	32	5.0	943228858

El archivo `movies.csv` contiene información sobre las películas, con los siguientes campos: `movieId`, `title` y `genres`. La columna de 'genres' contiene los géneros de la película, separados por '|'. En la tabla 3 se observan los primeros 5 registros de esta base de datos.

Cuadro 3: Ejemplo de las primeras filas de `movies.csv`.

<code>movieId</code>	<code>title</code>	<code>genres</code>
1	<i>Toy Story (1995)</i>	Adventure Animation Children Comedy Fantasy
2	<i>Jumanji (1995)</i>	Adventure Children Fantasy
3	<i>Grumpier Old Men (1995)</i>	Comedy Romance
4	<i>Waiting to Exhale (1995)</i>	Comedy Drama Romance
5	<i>Father of the Bride Part II (1995)</i>	Comedy

El análisis de los datos revela que `ratings.csv` contiene 32,000,204 registros y 4 columnas (856.52 MB), con calificaciones comprendidas entre 0,5 y 5, emitidas por 200,948 usuarios sobre 84,432 películas; el archivo carece de duplicados o valores nulos. Por otro lado, `movies.csv` posee 87,585 películas (4.14 MB) pero incluye 198 títulos duplicados que, tras identificarse, se eliminaron junto con sus correspondientes `movieId` en el conjunto de calificaciones, reduciendo así `ratings.csv` a 31,901,794 filas y garantizando la integridad entre ambos *datasets*. Cada usuario registra en promedio 160 calificaciones (mínimo 20), suficiente para descartar el problema de *cold-start* por usuarios nuevos. Este problema ocurre cuando el sistema carece de datos previos para inferir sus gustos y, en consecuencia, no puede personalizar las recomendaciones. Sin embargo, 160 calificaciones en promedio frente a las 84 mil películas indica que la matriz usuario-película resulta altamente dispersa.

En la figura 1, el diagrama de caja del número de calificaciones por usuario muestra una caja muy estrecha, indicando poca variación en el rango intercuartílico y revela valores atípicos extremos (más de 5,000 calificaciones). Estos valores pueden ser atribuibles

a críticos, usuarios muy frecuentes o comportamientos aleatorios; se opta por conservarlos porque su peso porcentual es ínfimo y ayudan a densificar la matriz sin sesgar las métricas globales.

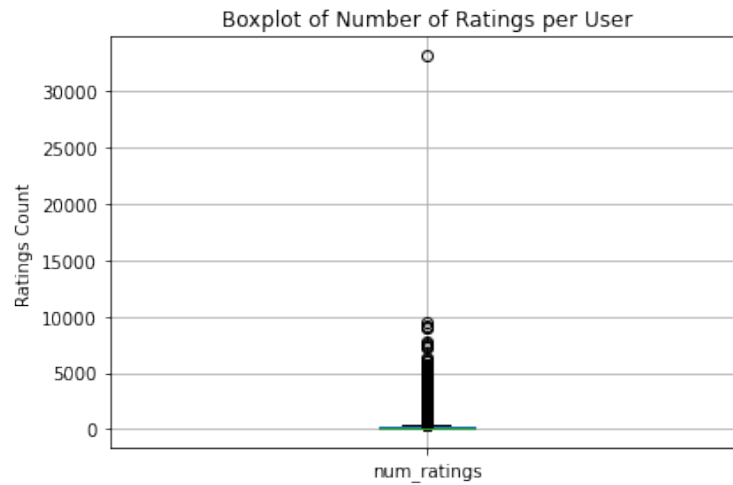


Figura 1: Distribución de número de Ratings por usuario.

Finalmente, la figura 2 muestra las diez películas más calificadas (p.ej., *The Shawshank Redemption*, *Forrest Gump*, *Pulp Fiction*), listado que sirve como recomendación inicial robusta para usuarios completamente nuevos y como punto de referencia para validar la coherencia del sistema de recomendación.

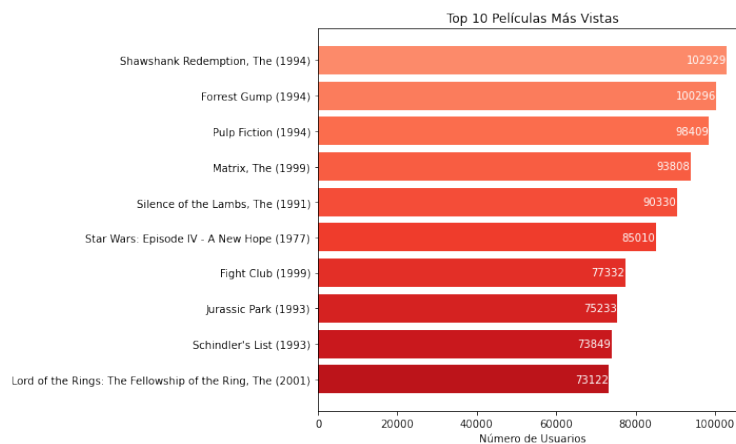


Figura 2: Top 10 películas más calificadas por usuarios.

6. Arquitectura

El problema aborda un volumen de datos propio de Big Data. El modelo ALS parte de una matriz de utilidad con 200,948 usuarios y 84,432 películas; aunque sólo se observan alrededor de 32 millones de calificaciones, el espacio potencial de interacciones

($200,948 \times 84,432$) supera los 17 mil millones de celdas. Durante cada iteración del algoritmo deben resolverse sistemas lineales y multiplicaciones distribuidas que leen todas las observaciones no nulas y mantienen k vectores latentes en memoria. Por ello, se traslada el cómputo a un cluster donde la memoria y la CPU se reparten entre nodos y el paralelismo de Spark puede usarse.

Para el almacenamiento se emplea un bucket de Amazon S3. Este servicio separa computación y datos: el clúster puede crearse o destruirse sin copiar archivos, mientras S3 garantiza durabilidad y ancho de banda prácticamente ilimitado [2]. Además, EMR integra S3 con Hadoop de forma nativa, de modo que los ejecutores de pySpark leen y escriben directamente sin pasos intermedios.

El cómputo distribuido reside en un clúster EMR compuesto por instancias EC2 m5.xlarge (4 vCPU, 16 GB RAM). La topología incluye un nodo primario que orquesta YARN y HDFS, un nodo core que aloja datos locales y ejecutores, y un nodo task dedicado exclusivamente a procesamiento; esta configuración puede ampliarse horizontalmente en minutos si el volumen crece [3]. El *stack* de software (Spark 3.5.4, Hadoop 3.4.1 y Hive 3.1.3) brinda tolerancia a fallos y capacidad de procesamiento distribuido. Sobre esta infraestructura, Jupyter Enterprise Gateway y JupyterHub permiten que se ejecuten notebooks de forma simultánea sin mover ni duplicar los datos.

Finalmente, al elegir Amazon Web Services (AWS), la arquitectura está preparada para escalar en volumen y velocidad. Si el catálogo crece a cientos de millones de ratings, basta con añadir nodos core para ampliar HDFS y redistribuir particiones debido a la flexibilidad de EC2 y EMR. Si las calificaciones llegan en tiempo real, Spark Structured Streaming puede ingerirlas desde Amazon Kinesis y recalcular factores en micro-lotes [4]. En conjunto, S3 como almacenamiento y EMR con Spark ofrecen la flexibilidad, la resiliencia y el control de costos que un sistema de recomendación industrial exige en la actualidad.

7. Resultados

7.1. Resultados

7.1.1. Filtrado colaborativo con ALS

Cuadro 4: Error cuadrático medio (RMSE) y tiempo de entrenamiento para cada valor de *rank*.

Rank (#factores)	RMSE
10	0.8430
20	0.8425
30	0.8427

El modelo ALS obtuvo su mejor rendimiento con *rank*= 20, alcanzando un *RMSE* de 0.8425, mientras que las ejecuciones con 10 y 30 factores produjeron 0.8430 y 0.8427, respectivamente. Las tres ejecuciones completas consumieron ~810 s, mientras que el

entrenamiento definitivo con la configuración óptima duró 254.9 s. La métrica obtenida es mejor que el resultado reportado en la literatura, donde con $\text{rank}=12$ se logró un *RMSE* de 0.9167 [1]; la diferencia supone una reducción relativa del error del 8,1 %. Además, el experimento se entrenó con 32 M de calificaciones, a comparación de 20 M utilizadas en el estudio citado y aun así converge a un error menor, lo que indica que la mayor densidad de datos y el uso de veinte factores latentes proporcionan una representación más rica sin incrementar de forma significativa aunque a un mayor costo computacional.

7.1.2. Filtrado basado en contenido

Cuadro 5: Top 10 películas más similares a *Toy Story* (1995) según la similitud coseno del modelo basado en contenido.

#	Título	Similitud
1	Toy Story 2 (1999)	0.9999999
2	Once Upon a Snowman (2020)	0.96229
3	Time Raiders (2016)	0.94436
4	One Piece: Curse of the Sacred Sword (2004)	0.94281
5	Who Framed Roger Rabbit? (1988)	0.93570
6	Goat Story With Cheese (2012)	0.93553
7	A Bug's Life (1998)	0.93553
8	Toy Story 4 (2019)	0.93553
9	The Snow Queen 2: Refreeze (2014)	0.92716
10	Mandie and the Secret Tunnel (2009)	0.92112

Como se mencionó anteriormente, evaluar cuantitativamente este recomendador basado en contenido resulta complejo y, por esta razón, se realiza a un análisis cualitativo. Al introducir *Toy Story* (1995) como película de referencia, el modelo devuelve como primeros resultados sus secuelas directas (*Toy Story 2* y *4*), títulos del mismo estudio o estética (*A Bug's Life*, *Lamp Life*) y otras animaciones familiares o de fantasía (*Once Upon a Snowman*, *DC League of Super-Pets*). También aparecen producciones de animación japonesa (*One Piece*, *Cardcaptor Sakura*), lo que demuestra que la representación *TF-IDF* identifica la etiqueta *Animation* y amplía la diversidad sin perder coherencia temática. La presencia de secuelas, títulos del mismo universo creativo y variedad controlada corrobora que el modelo genera recomendaciones plausibles aun sin una métrica numérica directa.

8. Conclusiones

Una de las conclusiones que se pueden deducir es que un número mayor de factores latentes en el filtrado colaborativo no implica un mejor resultado. Un número menor de factores latentes puede generar que la aproximación de la factorización de la matriz de utilidad no explique esta mismo. Por otro lado, un número alto puede generar problemas de sobre entrenamiento o estimación, por lo que la aproximación a la matriz de utilidad

es mejor, pero al momento de estimar calificaciones faltantes tenga un error mayor.

Mientras mayor sea el número de etiquetas y el tipo de estas, se obtendrán resultados más específicos acerca de la similitud entre películas. Esto se debe a que con un mayor número de etiquetas, la representación de cada película tendrá un grado de diferencia mayor a las otras, evitando distintivos iguales, con similitudes perfectas. Por esto, incluir el título de la película en las etiquetas es de gran utilidad para lograr esta diferenciación entre ítems.

Con lo desarrollado en este proyecto se brinda la posibilidad de extender la idea en otros ámbitos. La primera propuesta radica en implementar un modelo híbrido, es decir que involucre ambos métodos. Este podría ser a partir de un sistema de recomendación secuencial, donde se haga una primera selección por medio de filtrado basado en contenido y con el grupo de películas seleccionado realizar filtrado colaborativo para obtener la recomendación final. Con esto se puede reducir el las dimensiones de la matriz de utilidad.

Por otro lado, teniendo la matriz de similitud entre las películas se pueden realizar distintos esquemas para un sistema de recomendación. Un ejemplo de esto es recomendar con las primera n películas con mayor similaridad y realizar una ponderación de acuerdo a la similitud y calificación.

9. Disponibilidad del código

Todo el material necesario para reproducir los experimentos, como cuadernos de PySpark, datos de ejemplo y datos procesados guías, se encuentra en el repositorio de GitHub: <https://github.com/saracanom/Recomendandoando>.

10. Ejecución del plan

Lo primero es revisar qué de lo que estaba estipulado en el plan de ejecución en el anteproyecto. Para la entrega final se eliminaron algunas de las actividades que estaban propuestas en el anteproyecto, como: implementación de modelos híbridos y comparación para la selección del mejor modelo. Por otro lado, se debió agregar una actividad asociada al desarrollo de la arquitectura para el proyecto.

Uno de los imprevistos que se tuvieron al momento de la realización del proyecto radicó en que la capacidad de almacenamiento de un nodo, como Colab, no era suficiente para almacenar los resultados del sistema de recomendación. El error estaba en el momento en que se pretendía almacenar la matriz de utilidad. Por esto, se hizo transferencia a una arquitectura con AWS donde se pudiera realizar lo que se pretende. Por esto mismo, se tuvieron retrasos en el plan, mientras se estudiaba e implementaba esta arquitectura.

El desarrollo de una planeación para la realización de proyectos es fundamental en la estructuración y manejo del tiempo. Para el caso de este proyecto se obtienen múltiples

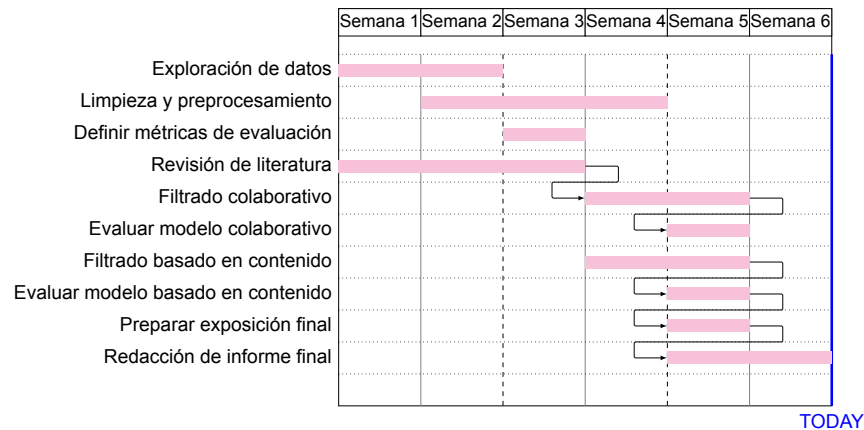


Figura 3: Cronograma del proyecto.

aprendizajes acerca de este. Lo primero es que mientras más específico sea la planeación, mejor se puede manejar el tiempo y controlar el progreso. Tener actividades muy generales hace que llevar registro del progreso sea más complicado. Si esta se dividen en sub-tareas y se ponen en la planeación, entonces es más probable cumplir con los objetivos.

Del mismo modo, mientras mayor sea el conocimiento sobre lo que se va a realizar en el proyecto, mejor estructurado quedará el plan de trabajo. De esta forma se pueden establecer fechas límites más acordes a la complejidad de las tareas, optimizando el uso del tiempo en actividades de complejidad baja y dedicando el tiempo necesario a las de mayor dificultad.

11. Implicaciones éticas

El desarrollo de un sistema de recomendación de películas basado en el conjunto de datos MovieLens implica diversas consideraciones éticas que deben ser tomadas en cuenta. En primer lugar, dado que el sistema influirá en las decisiones de los usuarios al seleccionar contenido, es fundamental garantizar que las recomendaciones sean diversas y no sesgadas para no limitar la exposición a nuevos géneros o perspectivas. Además, al utilizar datos históricos de calificaciones y etiquetas generadas por usuarios anónimos, es crucial respetar la privacidad de la información, asegurando que no se realicen inferencias que puedan comprometer la identidad de los usuarios.

Por otro lado, si el sistema fuera implementado en una plataforma real, su impacto en la industria cinematográfica y en la experiencia del usuario debe ser considerado. Un sistema sesgado podría favorecer ciertas películas o géneros, afectando la visibilidad de producciones independientes o de menor presupuesto. Asimismo, si el sistema presenta fallos en producción, como recomendaciones erróneas, podría generar frustración en los usuarios y afectar la confianza en la plataforma. Para reducir estos riesgos, el desarrollo del sistema debe incluir pruebas rigurosas para evitar sesgos en las recomendaciones y considerar mecanismos de revisión para garantizar la equidad y calidad del contenido.

sugerido.

Aunque la tecnología es una gran herramienta, no se puede despreciar el daño ambiental que esta puede llegar a generar. Con este proyecto, almacenar esta gran cantidad de información puede requerir una gran cantidad de recursos, generando un daño importante al medio ambiente.

12. Aspectos legales y comerciales

El sistema de recomendación de películas basado en MovieLens tiene un potencial comercial significativo, ya que los algoritmos de recomendación son clave en plataformas de streaming, comercio electrónico y servicios de contenido personalizado. Empresas como Netflix, Amazon Prime Video o Disney+ dependen de estos sistemas para mejorar la experiencia del usuario y aumentar la retención en sus plataformas. Dentro de una compañía, este tipo de modelos puede optimizar estrategias de contenido, segmentación de usuarios y publicidad dirigida. Por otro lado, los sistemas de recomendación han desplazado un rol en esta industria, pasando de asesores comerciales a estos sistemas. Esto tiene múltiples ventajas de las cuales destacan: el sesgo viene dado por los datos y no por factores comerciales o personales, un mayor conocimiento del usuario, posibilitando una mejor experiencia, rapidez para la consecución de la recomendación y un margen mayor para esta misma.

Desde el punto de vista legal, es fundamental considerar las licencias de uso de datos. La base de datos MovieLens está disponible para investigación bajo una licencia que prohíbe su uso comercial sin permiso de la Universidad de Minnesota. Esto significa que el sistema desarrollado no puede ser utilizado con fines comerciales sin la autorización explícita de los creadores del dataset. Además, cualquier publicación de resultados debe incluir la referencia correspondiente al conjunto de datos.

Referencias

- [1] M. F. Aljunid and D. H. Manjaiah. Movie recommender system based on collaborative filtering using apache spark. In V. E. Balas, N. Sharma, and A. Chakrabarti, editors, *Data Management, Analytics and Innovation*, pages 283–295, Singapore, 2019. Springer Singapore.
- [2] Amazon Web Services. Data protection in amazon s3, 2025. Consultado el 13 mayo 2025.
- [3] Amazon Web Services. Use amazon emr cluster scaling to adjust for changing workloads, 2025. Consultado el 13 mayo 2025.
- [4] Amazon Web Services. Using the spark structured streaming amazon kinesis data streams connector, 2025. Consultado el 13 mayo 2025.

- [5] F. M. Harper and J. A. Konstan. The movielens datasets: History and context. Online; accessed 2025-03-05, 2015. *ACM Transactions on Interactive Intelligent Systems (TiIS)*, 5(4), 19:1–19:19.
- [6] O. Kunde, O. Gaikwad, P. Kelgandre, R. Damodhar, and M. Swami. The movie recommendation system using content based filtering with tf-idf-vectorization and levenshtein distance. *International Journal of Advanced Research in Science, Communication and Technology*, pages 257–263, 05 2022.
- [7] A. Mc Ewen. Sistema de recomendación de anime. <https://github.com/amchp/anime-recommender>, 2024. Accedido: 13 may 2025.
- [8] Y. Zhou, D. Wilkinson, R. Schreiber, and R. Pan. Large-scale parallel collaborative filtering for the netflix prize. In *Algorithmic Aspects in Information and Management: 4th International Conference, AAIM 2008, Shanghai, China, June 23-25, 2008. Proceedings 4*, pages 337–348. Springer, 2008.